RA1 - Preparação dos dados de COVID-19 na Índia

Este notebook reúne a leitura, a avaliação da qualidade e a preparação dos dados.
Execute na ordem. Os três CSVs originais são as únicas entradas necessárias.

1. Definição do problema e perguntas analíticas

Queremos compreender a evolução dos registros de COVID-19, da testagem e da vacinação entre estados e territórios da Índia. Primeiro precisamos preparar bases com períodos diferentes, lacunas e nomes territoriais inconsistentes.

• Como os registros acumulados de casos, recuperações e óbitos evoluíram em cada estado no período disponível?

• Como o volume registrado de testes variou entre estados e ao longo do tempo?

• Como a administração da primeira e da segunda dose evoluiu nos estados durante 2021?

2. Fontes de dados

Os arquivos foram fornecidos como provenientes do conjunto COVID-19 in India , de Sudalai Rajkumar, distribuído no Kaggle. A data e a versão do download não foram informadas; a origem primária específica de cada arquivo não foi confirmada.

Arquivo	Formato	Principais campos
covid_19_india.csv	CSV UTF-8, separado por vírgulas	Data, território, confirmados, recuperados e óbitos
StatewiseTestingDetails.csv	CSV UTF-8, separado por vírgulas	Data, estado, amostras, negativos e positivos
covid_vaccine_statewise.csv	CSV UTF-8, separado por vírgulas	Data, estado, doses e recortes por imunizante, idade e sexo

São dados históricos. As quantidades de registros, colunas e amostras aparecem após a leitura. Todas as variáveis serão preservadas; o dicionário em docs/data_dictionary.md explica seus significados.

3. Ingestão dos dados

Vamos ler os valores como texto. Assim podemos conferir os dados antes de converter datas e números.

```
from pathlib import Path
import pandas as pd

# Os arquivos ficam na raiz; se estivermos em notebooks, subimos uma pasta.
pasta = Path("..") if Path.cwd().name == "notebooks" else Path(".")
arquivos = {
    "casos": "covid_19_india.csv",
    "testes": "StatewiseTestingDetails.csv",
    "vacinacao": "covid_vaccine_statewise.csv",
}

# Lendo como texto, sem trocar os marcadores de ausência por enquanto.
originais = {}
```

```
for nome, arquivo in arquivos.items():
    originais[nome] = pd.read_csv(
        pasta / arquivo, dtype="string", keep_default_na=False, encoding="utf-8-sig"
    )

# Conferindo o tamanho e as primeiras linhas de cada arquivo carregado.
for nome, df in originais.items():
    print(f"{arquivos[nome]}: {len(df)} linhas e {len(df.columns)} colunas")
    display(df.head(3))
```

```
covid_19_india.csv: 18110 linhas e 9 colunas
```

	Sno	Date	Time	State/UnionTerritory	ConfirmedIndianNational	ConfirmedForeignNational
0	1	2020-01-30	6:00 PM	Kerala	1	0
1	2	2020-01-31	6:00 PM	Kerala	1	0
2	3	2020-02-01	6:00 PM	Kerala	2	0

	Cured	Deaths	Confirmed
0	0	0	1
1	0	0	1
2	0	0	2

```
StatewiseTestingDetails.csv: 16336 linhas e 5 colunas
```

	Date	State	TotalSamples	Negative	Positive
0	2020-04-17	Andaman and Nicobar Islands	1403.0	1210	12.0
1	2020-04-24	Andaman and Nicobar Islands	2679.0		27.0
2	2020-04-27	Andaman and Nicobar Islands	2848.0		33.0

```
covid_vaccine_statewise.csv: 7845 linhas e 24 colunas
```

	Updated On	State	Total Doses Administered	Sessions	Sites	First Dose Administered
0	16/01/2021	India	48276.0	3455.0	2957.0	48276.0
1	17/01/2021	India	58604.0	8532.0	4954.0	58604.0
2	18/01/2021	India	99449.0	13611.0	6583.0	99449.0

	Second Dose Administered	Male (Doses Administered)	Female (Doses Administered)	Transgender (Doses Administered)	...	18-44 Years (Doses Administered)
0	0.0				...	
1	0.0				...	

	Second Dose Administered	Male (Doses Administered)	Female (Doses Administered)	Transgender (Doses Administered)	...	18-44 Years (Doses Administered)
2	0.0				...	

	45-60 Years (Doses Administered)	60+ Years (Doses Administered)	18-44 Years(Individuals Vaccinated)	45-60 Years(Individuals Vaccinated)	60+ Years(Individuals Vaccinated)	Male(Individuals Vaccinated)
0						23757.0
1						27348.0
2						41361.0

	Female(Individuals Vaccinated)	Transgender(Individuals Vaccinated)	Total Individuals Vaccinated
0	24517.0	2.0	48276.0
1	31252.0	4.0	58604.0
2	58083.0	5.0	99449.0

3 rows × 24 columns

4. Avaliação da qualidade dos dados

Vamos procurar valores vazios, linhas repetidas, datas e números inválidos.
Todos os campos foram lidos como texto; a conversão será feita depois.

As tabelas desta etapa usam rótulos em português apenas para facilitar a leitura. Os nomes das colunas dos arquivos continuam os mesmos.

```
# Usando os mesmos nomes curtos que já existem nos arquivos tratados.
mapa_colunas = {
    "casos": {
        "Sno": "sno",
        "Date": "date",
        "Time": "time",
        "State/UnionTerritory": "state",
        "ConfirmedIndianNational": "confirmed_indian_national",
        "ConfirmedForeignNational": "confirmed_foreign_national",
        "Cured": "cured",
        "Deaths": "deaths",
        "Confirmed": "confirmed"
    },
    "testes": {
        "Date": "date",
        "State": "state",
        "TotalSamples": "total_samples",
        "Negative": "negative",
        "Positive": "positive"
    },
    "vacinacao": {
        "Updated On": "date",
```

```
        "State": "state",
        "Total Doses Administered": "total_doses_administered",
        "Sessions": "sessions",
        " Sites ": "sites",
        "First Dose Administered": "first_dose_administered",
        "Second Dose Administered": "second_dose_administered",
        "Male (Doses Administered)": "male_doses_administered",
        "Female (Doses Administered)": "female_doses_administered",
        "Transgender (Doses Administered)": "transgender_doses_administered",
        " Covaxin (Doses Administered)": "covaxin_doses_administered",
        "CoviShield (Doses Administered)": "covi_shield_doses_administered",
        "Sputnik V (Doses Administered)": "sputnik_v_doses_administered",
        "AEFI": "aefi",
        "18-44 Years (Doses Administered)": "18_44_years_doses_administered",
        "45-60 Years (Doses Administered)": "45_60_years_doses_administered",
        "60+ Years (Doses Administered)": "60_plus_years_doses_administered",
        "18-44 Years(Individuals Vaccinated)": "18_44_years_individuals_vaccinated",
        "45-60 Years(Individuals Vaccinated)": "45_60_years_individuals_vaccinated",
        "60+ Years(Individuals Vaccinated)": "60_plus_years_individuals_vaccinated",
        "Male(Individuals Vaccinated)": "male_individuals_vaccinated",
        "Female(Individuals Vaccinated)": "female_individuals_vaccinated",
        "Transgender(Individuals Vaccinated)": "transgender_individuals_vaccinated",
        "Total Individuals Vaccinated": "total_individuals_vaccinated"
    }
}

# Estes rótulos servem só para mostrar as tabelas em português.
rotulos_colunas = {
    "sno": "Número do registro",
    "date": "Data",
    "time": "Hora",
    "state": "Estado ou território",
    "confirmed_indian_national": "Confirmados de nacionalidade indiana",
    "confirmed_foreign_national": "Confirmados de nacionalidade estrangeira",
    "cured": "Recuperados",
    "deaths": "Óbitos",
    "confirmed": "Confirmados",
    "total_samples": "Total de amostras",
    "negative": "Resultados negativos",
    "positive": "Resultados positivos",
    "total_doses_administered": "Total de doses aplicadas",
    "sessions": "Sessões de vacinação",
    "sites": "Locais de vacinação",
    "first_dose_administered": "Primeiras doses aplicadas",
    "second_dose_administered": "Segundas doses aplicadas",
    "male_doses_administered": "Doses aplicadas em homens",
    "female_doses_administered": "Doses aplicadas em mulheres",
    "transgender_doses_administered": "Doses aplicadas em pessoas transgênero",
    "covaxin_doses_administered": "Doses da Covaxin",
    "covi_shield_doses_administered": "Doses da CoviShield",
    "sputnik_v_doses_administered": "Doses da Sputnik V",
    "aefi": "Eventos adversos após vacinação",
    "18_44_years_doses_administered": "Doses na faixa de 18 a 44 anos",
```

```
    "45_60_years_doses_administered": "Doses na faixa de 45 a 60 anos",
    "60_plus_years_doses_administered": "Doses na faixa de 60 anos ou mais",
    "18_44_years_individuals_vaccinated": "Pessoas vacinadas de 18 a 44 anos",
    "45_60_years_individuals_vaccinated": "Pessoas vacinadas de 45 a 60 anos",
    "60_plus_years_individuals_vaccinated": "Pessoas vacinadas de 60 anos ou mais",
    "male_individuals_vaccinated": "Homens vacinados",
    "female_individuals_vaccinated": "Mulheres vacinadas",
    "transgender_individuals_vaccinated": "Pessoas transgênero vacinadas",
    "total_individuals_vaccinated": "Total de pessoas vacinadas"
}
nomes_bases = {"casos": "Casos", "testes": "Testes", "vacinacao": "Vacinação"}

# Estes textos representam valores que não foram informados.
marcadores_ausentes = ["", "-", "NA", "N/A", "NaN", "nan", "null"]
formatos_data = {"casos": "%Y-%m-%d", "testes": "%Y-%m-%d", "vacinacao": "%d/%m/%Y"}
avaliacao = {}
ausencias_antes = {}
numericas = {}

for nome, original in originais.items():
    # Conferindo se o arquivo tem as colunas esperadas.
    if list(original.columns) != list(mapa_colunas[nome]):
        raise ValueError(f"Colunas inesperadas em {nome}; confira o arquivo de entrada.")
    # Trabalhando numa cópia, sem alterar o que foi carregado.
    df = original.rename(columns=mapa_colunas[nome]).copy()
    # Ignorando espaços nas pontas e reconhecendo os valores vazios.
    for coluna in df:
        df[coluna] = df[coluna].str.strip().replace(marcadores_ausentes, pd.NA)
    avaliacao[nome] = df
    ausencias_antes[nome] = df.isna().sum()
    # As outras colunas são contagens; data, estado e hora ficam de fora.
    numericas[nome] = [coluna for coluna in df if coluna not in ["date", "state", "time"]]
    print(nomes_bases[nome])
    # Contando os vazios de cada coluna, com nomes traduzidos só na tabela.
    tabela = pd.DataFrame({
        "Tipo de Dado": df.dtypes.astype(str).replace({"string": "Texto"}),
        "Valores Ausentes": df.isna().sum(),
    }).rename(index=rotulos_colunas)
    tabela.index.name = "Coluna"
    display(tabela)
    # Conferindo repetições de linhas inteiras e de estado com data.
    print("Linhas idênticas repetidas:", original.duplicated().sum())
    print("Linhas com estado/data repetidos:", df.duplicated(["state", "date"], keep=False).sum())
    # Nomes de lugares são mantidos; traduzimos apenas categorias gerais.
    categorias = df["state"].replace({
        "India": "Índia", "Unassigned": "Não atribuído",
        "Cases being reassigned to states": "Casos em redistribuição entre estados",
    })
    print("Estados, territórios e outras categorias:", sorted(categorias.dropna().unique()))
```

```
Casos
```

	Tipo de Dado	Valores Ausentes
Coluna		
Número do registro	Texto	0
Data	Texto	0
Hora	Texto	0
Estado ou território	Texto	0
Confirmados de nacionalidade indiana	Texto	17664
Confirmados de nacionalidade estrangeira	Texto	17664
Recuperados	Texto	0
Óbitos	Texto	0
Confirmados	Texto	0

```
Linhas idênticas repetidas: 0
Linhas com estado/data repetidos: 0
Estados, territórios e outras categorias: ['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Bihar****',
    'Casos em redistribuição entre estados', 'Chandigarh', 'Chhattisgarh', 'Dadra and Nagar Haveli', 'Dadra and Nagar Haveli and Daman and Diu',
    'Daman & Diu', 'Delhi', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Himanchal Pradesh', 'Jammu and Kashmir', 'Jharkhand', 'Karanataka',
     'Karnataka', 'Kerala', 'Ladakh', 'Lakshadweep', 'Madhya Pradesh', 'Madhya Pradesh***', 'Maharashtra', 'Maharashtra***', 'Manipur',
    'Meghalaya', 'Mizoram', 'Nagaland', 'Não atribuído', 'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana',
    'Telengana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']
Testes
```

	Tipo de Dado	Valores Ausentes
Coluna		
Data	Texto	0
Estado ou território	Texto	0
Total de amostras	Texto	0
Resultados negativos	Texto	9368
Resultados positivos	Texto	10674

```
Linhas idênticas repetidas: 1
Linhas com estado/data repetidos: 2
Estados, territórios e outras categorias: ['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh',
    'Chhattisgarh', 'Dadra and Nagar Haveli and Daman and Diu', 'Delhi', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir',
    'Jharkhand', 'Karnataka', 'Kerala', 'Ladakh', 'Lakshadweep', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland',
    'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal']

Vacinação
```

	Tipo de Dado	Valores Ausentes
Coluna		
Data	Texto	0
Estado ou território	Texto	0
Total de doses aplicadas	Texto	224
Sessões de vacinação	Texto	224
Locais de vacinação	Texto	224
Primeiras doses aplicadas	Texto	224
Segundas doses aplicadas	Texto	224
Doses aplicadas em homens	Texto	384
Doses aplicadas em mulheres	Texto	384
Doses aplicadas em pessoas transgênero	Texto	384
Doses da Covaxin	Texto	224
Doses da CoviShield	Texto	224
Doses da Sputnik V	Texto	4850
Eventos adversos após vacinação	Texto	2407
Doses na faixa de 18 a 44 anos	Texto	6143
Doses na faixa de 45 a 60 anos	Texto	6143
Doses na faixa de 60 anos ou mais	Texto	6143
Pessoas vacinadas de 18 a 44 anos	Texto	4112
Pessoas vacinadas de 45 a 60 anos	Texto	4111
Pessoas vacinadas de 60 anos ou mais	Texto	4111
Homens vacinados	Texto	7685
Mulheres vacinadas	Texto	7685
Pessoas transgênero vacinadas	Texto	7685
Total de pessoas vacinadas	Texto	1926

```
Linhas idênticas repetidas: 0
Linhas com estado/data repetidos: 0
Estados, territórios e outras categorias: ['Andaman and Nicobar Islands', 'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chandigarh',
    'Chhattisgarh', 'Dadra and Nagar Haveli and Daman and Diu', 'Delhi', 'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jammu and Kashmir',
    'Jharkhand', 'Karnataka', 'Kerala', 'Ladakh', 'Lakshadweep', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya', 'Mizoram', 'Nagaland',
    'Odisha', 'Puducherry', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West Bengal',
     'Índia']

# Conferindo se cada data pode ser lida no formato do seu arquivo.
```

```
diagnostico = []
for nome, df in avaliacao.items():
    datas = pd.to_datetime(df["date"], format=formatos_data[nome], errors="coerce")
    diagnostico.append({"base": nome, "campo": "date", "invalidos": int(datas.isna().sum())})

    # Ausência é permitida. O problema é texto inválido, número negativo ou fracionado.
    for coluna in numericas[nome]:
        numeros = pd.to_numeric(df[coluna], errors="coerce")
        invalidos = df[coluna].notna() & (
            numeros.isna() | numeros.lt(0) | numeros.ge(2**63) | numeros.mod(1).ne(0)
        )
        diagnostico.append({"base": nome, "campo": coluna, "invalidos": int(invalidos.sum())})

# Traduzindo a apresentação, sem mudar as verificações ou os valores.
tabela_diagnostico = pd.DataFrame(diagnostico)
tabela_diagnostico["base"] = tabela_diagnostico["base"].map(nomes_bases)
tabela_diagnostico["campo"] = tabela_diagnostico["campo"].map(rotulos_colunas)
display(tabela_diagnostico.rename(columns={
    "base": "Base", "campo": "Coluna", "invalidos": "Valores Inválidos"
}))
```

	Base	Coluna	Valores Inválidos
0	Casos	Data	0
1	Casos	Número do registro	0
2	Casos	Confirmados de nacionalidade indiana	0
3	Casos	Confirmados de nacionalidade estrangeira	0
4	Casos	Recuperados	0
5	Casos	Óbitos	0
6	Casos	Confirmados	0
7	Testes	Data	0
8	Testes	Total de amostras	0
9	Testes	Resultados negativos	0
10	Testes	Resultados positivos	0
11	Vacinação	Data	0
12	Vacinação	Total de doses aplicadas	0
13	Vacinação	Sessões de vacinação	0
14	Vacinação	Locais de vacinação	0
15	Vacinação	Primeiras doses aplicadas	0
16	Vacinação	Segundas doses aplicadas	0
17	Vacinação	Doses aplicadas em homens	0

	Base	Coluna	Valores Inválidos
18	Vacinação	Doses aplicadas em mulheres	0
19	Vacinação	Doses aplicadas em pessoas transgênero	0
20	Vacinação	Doses da Covaxin	0
21	Vacinação	Doses da CoviShield	0
22	Vacinação	Doses da Sputnik V	0
23	Vacinação	Eventos adversos após vacinação	0
24	Vacinação	Doses na faixa de 18 a 44 anos	0
25	Vacinação	Doses na faixa de 45 a 60 anos	0
26	Vacinação	Doses na faixa de 60 anos ou mais	0
27	Vacinação	Pessoas vacinadas de 18 a 44 anos	0
28	Vacinação	Pessoas vacinadas de 45 a 60 anos	0
29	Vacinação	Pessoas vacinadas de 60 anos ou mais	0
30	Vacinação	Homens vacinados	0
31	Vacinação	Mulheres vacinadas	0
32	Vacinação	Pessoas transgênero vacinadas	0
33	Vacinação	Total de pessoas vacinadas	0

As contagens devem ser inteiras e não negativas. Valores vazios continuam vazios.

Depois de organizar nomes e datas, também verificaremos quedas dos totais e diferenças entre colunas. Esses alertas serão mantidos, pois podem ser revisões dos boletins. Um valor alto, sozinho, não é motivo para excluir uma linha.

5. Limpeza e preparação dos dados

Linhas repetidas e nomes de estados

Removemos apenas linhas idênticas. Guardamos a linha de origem e o nome recebido para poder consultar o registro original. Corrigimos as grafias já identificadas, sem juntar territórios históricos diferentes.

```
# Corrigindo apenas as grafias que encontramos nos arquivos.
correcoes_estados = {
    "Karanataka": "Karnataka", "Himanchal Pradesh": "Himachal Pradesh",
    "Telengana": "Telangana", "Bihar****": "Bihar",
    "Madhya Pradesh***": "Madhya Pradesh", "Maharashtra***": "Maharashtra",
    "Daman & Diu": "Daman and Diu",
}
preparados = {}
duplicatas_removidas = {}
for nome, original in originais.items():
```

```
    # Procurando linhas iguais em todas as colunas originais.
    repetidas = original.duplicated()
    duplicatas_removidas[nome] = int(repetidas.sum())
    if repetidas.any():
        print("Duplicatas removidas:", nome)
        display(original.loc[repetidas])
    df = original.rename(columns=mapa_colunas[nome]).copy()
    # Guardando a linha de origem e o nome antes da correção.
    df.insert(0, "source_row", range(2, len(df) + 2))
    df["state_original"] = df["state"]
    # Mantendo a primeira ocorrência de cada linha idêntica.
    df = df.loc[~repetidas].copy()
    # Tirando espaços nas pontas e marcando ausências, sem colocar zero.
    for coluna in mapa_colunas[nome].values():
        df[coluna] = df[coluna].str.strip().replace(marcadores_ausentes, pd.NA)
    df["state"] = df["state"].replace(correcoes_estados)
    # Separando o total do país, os estados e os registros sem estado definido.
    df["geographic_level"] = "state_or_union_territory"
    df.loc[df["state"].eq("India"), "geographic_level"] = "national"
    df.loc[df["state"].isin(["Unassigned", "Cases being reassigned to states"]), "geographic_level"] = "unassigned"
    preparados[nome] = df
    # Mostrando quais nomes foram corrigidos.
    alterados = df["state_original"].ne(df["state"]).fillna(False)
    print(nome, "- registros com nome padronizado:", int(alterados.sum()))
    display(df.loc[alterados, ["state_original", "state"]].drop_duplicates())
```

```
casos - registros com nome padronizado: 433
```

	state_original	state
32	Telengana	Telangana
2890	Daman & Diu	Daman and Diu
15846	Bihar****	Bihar
17049	Madhya Pradesh***	Madhya Pradesh
17294	Himanchal Pradesh	Himachal Pradesh
17297	Karanataka	Karnataka
17338	Maharashtra***	Maharashtra

```
Duplicatas removidas: testes
```

	Date	State	TotalSamples	Negative	Positive
14863	2020-04-05	Uttar Pradesh	5255.0	4796	278.0

```
testes - registros com nome padronizado: 0
```

	state_original	state

```
vacinacao - registros com nome padronizado: 0
```

	state_original	state

Datas e números

Casos e testes usam ano-mês-dia; vacinação usa dia/mês/ano. Depois da conversão, verificamos se cada estado tem apenas um registro por data.

As contagens usam Int64 , que aceita números inteiros e valores vazios. Removemos apenas a parte decimal formada por zeros, como em 12.0 . Não arredondamos valores.

```
# Lendo as datas com o formato de cada base.
for nome, df in preparados.items():
    datas = pd.to_datetime(df["date"], format=formatos_data[nome], errors="coerce")
    if datas.isna().any() or df["state"].isna().any():
        raise ValueError(f"Data ou território inválido em {nome}; revise antes de exportar.")
    # Salvando todas as datas no formato ano-mês-dia.
    df["date"] = datas.dt.strftime("%Y-%m-%d").astype("string")
    # Não escolhemos uma linha ao acaso se estado e data se repetirem.
    if df.duplicated(["state", "date"]).any():
        raise ValueError(f"Chaves repetidas após padronização em {nome}; revise antes de exportar.")

    for coluna in numericas[nome]:
        # Valores como "12.0" viram "12"; não arredondamos frações.
        texto = df[coluna].str.replace(r"\.0+$", "", regex=True)
        # Conferindo se sobraram apenas dígitos nos valores preenchidos.
        if not texto.dropna().str.fullmatch(r"[0-9]+").all():
            raise ValueError(f"Contagem inválida em {nome}/{coluna}; revise antes de exportar.")
        try:
            # Int64 aceita inteiros e vazios; a conversão direta não perde dígitos.
            df[coluna] = texto.astype("Int64")
        except (ValueError, TypeError, OverflowError) as erro:
            raise ValueError(f"Contagem fora do intervalo Int64 em {nome}/{coluna}.") from erro
    preparados[nome] = df

# Padronizando a hora sem presumir um fuso horário.
horas_originais = preparados["casos"]["time"].str.upper()
horas = pd.Series(pd.NA, index=horas_originais.index, dtype="string")
# Tentando os formatos de hora usados na fonte.
for formato in ["%I:%M %p", "%H:%M", "%H:%M:%S"]:
    convertidas = pd.to_datetime(horas_originais, format=formato, errors="coerce")
    # Preenchendo apenas as horas que ainda não foram reconhecidas.
    horas = horas.fillna(convertidas.dt.strftime("%H:%M:%S"))
if horas.isna().any():
    raise ValueError("Hora inválida em casos; revise antes de exportar.")
preparados["casos"]["time"] = horas

# Organizando os registros por estado e data.
for nome, df in preparados.items():
    preparados[nome] = df.sort_values(["state", "date", "source_row"]).reset_index(drop=True)
```

Alertas que serão mantidos

Comparamos cada registro com o anterior do mesmo estado. Uma queda pode ser uma revisão do boletim, então não alteramos o valor.

Conferimos recuperados e óbitos em relação aos confirmados, além das primeiras e segundas doses em relação ao total. Não exigimos que positivos + negativos seja igual ao total de amostras, pois as contagens podem ter coberturas diferentes.

```
# Escolhendo as colunas de totais acumulados que vamos conferir.
acumuladas = {
    "casos": ["confirmed", "cured", "deaths"],
    "testes": ["total_samples"],
    "vacinacao": ["total_doses_administered", "first_dose_administered", "second_dose_administered"],
}
quedas = []
for nome, df in preparados.items():
    for coluna in acumuladas[nome]:
        # Comparando com o registro anterior do mesmo estado.
        anterior = df.groupby("state")[coluna].shift()
        # Se algum valor estiver vazio, não contamos como queda.
        queda = df[coluna].lt(anterior).fillna(False)
        quedas.append({"base": nome, "campo": coluna, "quedas": int(queda.sum())})
display(pd.DataFrame(quedas))

casos = preparados["casos"]
vacinacao = preparados["vacinacao"]
# Conferindo relações entre colunas, sem corrigir os valores automaticamente.
inconsistencias = {
    "Recuperados > confirmados": casos["cured"] > casos["confirmed"],
    "Óbitos > confirmados": casos["deaths"] > casos["confirmed"],
    "Recuperados + óbitos > confirmados": casos["cured"] + casos["deaths"] > casos["confirmed"],
    "Primeira + segunda dose diferente do total": vacinacao["first_dose_administered"] + vacinacao["second_dose_administered"] !=
    vacinacao["total_doses_administered"],
    "Segunda dose > primeira dose": vacinacao["second_dose_administered"] > vacinacao["first_dose_administered"],
}
display(pd.DataFrame([
    {"verificacao": regra, "ocorrencias": int(mascara.fillna(False).sum())}
    for regra, mascara in inconsistencias.items()
]))
# Mostrando os menores e maiores valores, sem excluir extremos.
for nome, df in preparados.items():
    print(nome, "- intervalos observados, sem excluir extremos")
    medidas = [coluna for coluna in numericas[nome] if coluna != "sno"]
    display(df[medidas].agg(["min", "max"]).T)
```

	base	campo	quedas
0	casos	confirmed	49
1	casos	cured	2
2	casos	deaths	1
3	testes	total_samples	23
4	vacinacao	total_doses_administered	57
5	vacinacao	first_dose_administered	64

	base	campo	quedas
6	vacinacao	second_dose_administered	3

	verificacao	ocorrencias
0	Recuperados > confirmados	0
1	Óbitos > confirmados	0
2	Recuperados + óbitos > confirmados	7
3	Primeira + segunda dose diferente do total	0
4	Segunda dose > primeira dose	0

`casos - intervalos observados, sem excluir extremos`

	min	max
confirmed_indian_national	0	177
confirmed_foreign_national	0	14
cured	0	6159676
deaths	0	134201
confirmed	0	6363442

`testes - intervalos observados, sem excluir extremos`

	min	max
total_samples	58	67897856
negative	0	83561026
positive	0	1638961

`vacinacao - intervalos observados, sem excluir extremos`

	min	max
total_doses_administered	7	513228400
sessions	0	35010311
sites	0	73933
first_dose_administered	7	400150406
second_dose_administered	0	113077994
male_doses_administered	0	270163622
female_doses_administered	2	239518609

	min	max
transgender_doses_administered	0	98275
covaxin_doses_administered	0	62367416
covi_shield_doses_administered	7	446825051
sputnik_v_doses_administered	0	588039
aefi	0	26542
18_44_years_doses_administered	26624	224330364
45_60_years_doses_administered	16815	166757453
60_plus_years_doses_administered	9994	118692689
18_44_years_individuals_vaccinated	1059	92243148
45_60_years_individuals_vaccinated	1136	90968877
60_plus_years_individuals_vaccinated	558	67310981
male_individuals_vaccinated	23757	134941971
female_individuals_vaccinated	24517	115668447
transgender_individuals_vaccinated	2	46462
total_individuals_vaccinated	7	250656880

6. Conjunto de dados resultante

As três bases continuam separadas, com todas as variáveis e três colunas extras que indicam a origem do registro e o tipo de território.
Vamos conferir os resultados da limpeza antes de salvar os CSVs.

```
resumo = []
for nome, df in preparados.items():
    # Cada linha precisa ter estado e data, sem repetição dessa combinação.
    assert not df[["state", "date"]].isna().any().any(), f"Chave ausente em {nome}"
    assert not df.duplicated(["state", "date"]).any(), f"Chave repetida em {nome}"
    # A quantidade só pode diminuir pelas duplicatas que removemos.
    assert len(df) + duplicatas_removidas[nome] == len(originais[nome])
    # As contagens preenchidas precisam continuar inteiras e não negativas.
    assert df[numericas[nome]].ge(0).fillna(True).all().all()
    assert all(str(df[coluna].dtype) == "Int64" for coluna in numericas[nome])
    # Reunindo tamanho e período de cada base preparada.
    resumo.append({
        "base": nome, "entrada": len(originais[nome]), "saida": len(df),
        "duplicatas_removidas": duplicatas_removidas[nome], "colunas": len(df.columns),
        "inicio": df["date"].min(), "fim": df["date"].max(),
    })
    print(nome)
    # Conferindo tipos e vazios antes e depois da limpeza desta execução.
```

```
    display(pd.DataFrame({
        "tipo_final": df.dtypes.astype(str),
        "ausentes_antes": ausencias_antes[nome].reindex(df.columns).astype("Int64"),
        "ausentes_depois": df.isna().sum(),
    }))
    display(df.head(3))
display(pd.DataFrame(resumo))
print("Verificações finais aprovadas. Dados prontos para salvar.")
```

```
casos
```

	tipo_final	ausentes_antes	ausentes_depois
source_row	int64	<NA>	0
sno	Int64	0	0
date	string	0	0
time	string	0	0
state	string	0	0
confirmed_indian_national	Int64	17664	17664
confirmed_foreign_national	Int64	17664	17664
cured	Int64	0	0
deaths	Int64	0	0
confirmed	Int64	0	0
state_original	string	<NA>	0
geographic_level	str	<NA>	0

	source_row	sno	date	time	state	confirmed_indian_national
0	367	366	2020-03-26	18:00:00	Andaman and Nicobar Islands	1
1	394	393	2020-03-27	10:00:00	Andaman and Nicobar Islands	1
2	422	421	2020-03-28	18:00:00	Andaman and Nicobar Islands	6

	confirmed_foreign_national	cured	deaths	confirmed	state_original	geographic_level
0	0	0	0	1	Andaman and Nicobar Islands	state_or_union_territory
1	0	0	0	1	Andaman and Nicobar Islands	state_or_union_territory
2	0	0	0	6	Andaman and Nicobar Islands	state_or_union_territory

```
testes
```

	tipo_final	ausentes_antes	ausentes_depois
source_row	int64	<NA>	0
date	string	0	0
state	string	0	0
total_samples	Int64	0	0
negative	Int64	9368	9368
positive	Int64	10674	10674
state_original	string	<NA>	0
geographic_level	str	<NA>	0

	source_row	date	state	total_samples	negative	positive
0	2	2020-04-17	Andaman and Nicobar Islands	1403	1210	12
1	3	2020-04-24	Andaman and Nicobar Islands	2679	<NA>	27
2	4	2020-04-27	Andaman and Nicobar Islands	2848	<NA>	33

	state_original	geographic_level
0	Andaman and Nicobar Islands	state_or_union_territory
1	Andaman and Nicobar Islands	state_or_union_territory
2	Andaman and Nicobar Islands	state_or_union_territory

`vacinacao`

	tipo_final	ausentes_antes	ausentes_depois
source_row	int64	<NA>	0
date	string	0	0
state	string	0	0
total_doses_administered	Int64	224	224
sessions	Int64	224	224
sites	Int64	224	224
first_dose_administered	Int64	224	224
second_dose_administered	Int64	224	224
male_doses_administered	Int64	384	384
female_doses_administered	Int64	384	384
transgender_doses_administered	Int64	384	384

	tipo_final	ausentes_antes	ausentes_depois
covaxin_doses_administered	Int64	224	224
covi_shield_doses_administered	Int64	224	224
sputnik_v_doses_administered	Int64	4850	4850
aefi	Int64	2407	2407
18_44_years_doses_administered	Int64	6143	6143
45_60_years_doses_administered	Int64	6143	6143
60_plus_years_doses_administered	Int64	6143	6143
18_44_years_individuals_vaccinated	Int64	4112	4112
45_60_years_individuals_vaccinated	Int64	4111	4111
60_plus_years_individuals_vaccinated	Int64	4111	4111
male_individuals_vaccinated	Int64	7685	7685
female_individuals_vaccinated	Int64	7685	7685
transgender_individuals_vaccinated	Int64	7685	7685
total_individuals_vaccinated	Int64	1926	1926
state_original	string	<NA>	0
geographic_level	str	<NA>	0

	source_row	date	state	total_doses_administered	sessions	sites
0	214	2021-01-16	Andaman and Nicobar Islands	23	2	2
1	215	2021-01-17	Andaman and Nicobar Islands	23	2	2
2	216	2021-01-18	Andaman and Nicobar Islands	42	9	2

	first_dose_administered	second_dose_administered	male_doses_administered	female_doses_administered	...	60_plus_years_doses_administered
0	23	0	12	11	...	<NA>
1	23	0	12	11	...	<NA>
2	42	0	29	13	...	<NA>

	18_44_years_individuals_vaccinated	45_60_years_individuals_vaccinated	60_plus_years_individuals_vaccinated	male_individuals_vaccinated	female_individuals_vaccinated	transgender_individuals_vaccinated
0	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
1	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>
2	<NA>	<NA>	<NA>	<NA>	<NA>	<NA>

	total_individuals_vaccinated	state_original	geographic_level
0	23	Andaman and Nicobar Islands	state_or_union_territory
1	23	Andaman and Nicobar Islands	state_or_union_territory
2	42	Andaman and Nicobar Islands	state_or_union_territory

3 rows × 27 columns

	base	entrada	saida	duplicatas_removidas	colunas	inicio
0	casos	18110	18110	0	12	2020-01-30
1	testes	16336	16335	1	8	2020-04-01
2	vacinacao	7845	7845	0	27	2021-01-16

	fim
0	2021-08-11
1	2021-08-10
2	2021-08-16

```
Verificações finais aprovadas. Dados prontos para salvar.
```

O valor vazio em “ausentes antes” nos três campos adicionados indica que eles não existiam na fonte.

Os arquivos abaixo são substituídos a cada execução. Os originais nunca são sobrescritos. CSV não guarda tipos: ao reler, informe Int64 para contagens e converta a data com o formato %Y-%m-%d .

No Colab, abra o painel Arquivos , entre em data/processed e baixe os três CSVs. Para guardar as saídas do notebook, use Arquivo → Fazer download → Fazer download do .ipynb .

```
# Criando a pasta de saída caso ela ainda não exista.
destino = pasta / "data" / "processed"
destino.mkdir(parents=True, exist_ok=True)
for nome, df in preparados.items():
    arquivo_saida = destino / f"{nome}.csv"
    # Salvando sem o índice do Pandas; valores vazios continuam vazios.
    df.to_csv(arquivo_saida, index=False, encoding="utf-8", lineterminator="\n")
    print(f"Salvo: data/processed/{nome}.csv - {len(df)} registros")
```

```
Salvo: data/processed/casos.csv - 18110 registros

Salvo: data/processed/testes.csv - 16335 registros

Salvo: data/processed/vacinacao.csv - 7845 registros
```

RA2 - Processamento de Dados em Streaming

1. Objetivo da análise

Vamos acompanhar novas amostras de testes por estado e por minuto , relacionadas à pergunta do RA1 sobre a evolução da testagem na Índia.
Usaremos Maharashtra e Karnataka, estados presentes na base de testes.

Todos os 9 registros abaixo são artificiais, criados apenas para esta demonstração acadêmica. Não são dados reais nem serão acrescentados aos CSVs do RA1. Cada registro representa um pequeno envio de novas amostras, e não o total acumulado TotalSamples da base original.

2. Fonte e processamento dos dados

Python escreve três arquivos JSON, um por lote, em momentos distintos da execução. O próximo arquivo só chega depois que o Spark processa o anterior. readStream acompanha a pasta e usa um schema explícito com horário, estado e quantidade de novas amostras.

Preparação: no VS Code, instale requirements.txt no kernel selecionado e use Java 17 (JAVA_HOME apontando para o JDK). No Colab, envie os três CSVs e execute %pip install pyspark==3.5.7 em uma célula antes desta seção, caso necessário. Use Java 17 também no Colab. Reinicie o kernel após instalar dependências. No Windows, o Hadoop pode exigir componentes nativos; nesse caso, use o mesmo notebook no VS Code com WSL ou no Colab.

```
import json
import tempfile
from uuid import uuid4
from pyspark.sql import SparkSession, functions as F, types as T

# Criando a sessão do Spark para a pequena demonstração local.
spark = (SparkSession.builder.master("local[2]")
         .appName("RA2 - Testagem simulada")
         .config("spark.sql.shuffle.partitions", "2")
         .config("spark.sql.session.timeZone", "UTC")
         .getOrCreate())
spark.sparkContext.setLogLevel("ERROR")

# Definindo os campos que vamos receber nos arquivos JSON.
schema_ra2 = T.StructType([
    T.StructField("horario", T.TimestampType(), False),
    T.StructField("estado", T.StringType(), False),
    T.StructField("novas_amostras", T.IntegerType(), False),
])

# Os horários são fictícios; UTC serve apenas para manter a reprodução igual.
def registro(hora, estado, quantidade):
    return {"horario": f"2021-08-01T{hora}Z",
            "estado": estado, "novas_amostras": quantidade}

lotes_ra2 = [
    [registro("10:00:05", "Maharashtra", 10),
     registro("10:00:20", "Karnataka", 8),
     registro("10:00:40", "Maharashtra", 5)],
    [registro("10:00:50", "Karnataka", 7),
```

```
     registro("10:01:00", "Maharashtra", 12),
     registro("10:01:15", "Karnataka", 6)],
    [registro("10:01:30", "Maharashtra", 3),
     registro("10:01:45", "Karnataka", 4),
     registro("10:02:00", "Maharashtra", 9)],
]

# Conferindo se os estados escolhidos existem nos dados preparados do RA1.
assert {"Maharashtra", "Karnataka"} <= set(preparados["testes"]["state"])
print("Simulação preparada: 3 lotes, com 3 registros artificiais cada.")
```

```
Simulação preparada: 3 lotes, com 3 registros artificiais cada.
```

3. Janelas temporais

Usamos uma janela Tumbling de 1 minuto , sem sobreposição, e a soma de novas_amostras por estado. A janela usa o horário do registro, não o horário em que o arquivo chegou. O intervalo inclui o início e exclui o fim: 10:01:00 já pertence à janela 10:01-10:02.

A consulta usa writeStream com saída em memória e modo complete , mostrando todos os totais atualizados. É suficiente para estes 9 registros. Não usamos watermark nem descartamos janelas nesta demonstração curta. processAllAvailable() aguarda o processamento antes de mostrar cada tabela; não há espera fixa com sleep() .

4. Resultados e interpretação

Execute a próxima célula para ver as três tabelas, na ordem de chegada. A pasta temporária e o checkpoint são novos a cada execução; os arquivos entram por renomeação depois da escrita completa. A consulta é encerrada mesmo se ocorrer um erro.

```
# Valores calculados à mão para conferir cada etapa da simulação.
# Cada item contém: início da janela, estado e total de novas amostras.
esperados_ra2 = [
    [("10:00:00", "Karnataka", 8), ("10:00:00", "Maharashtra", 15)],
    [("10:00:00", "Karnataka", 15), ("10:00:00", "Maharashtra", 15),
     ("10:01:00", "Karnataka", 6), ("10:01:00", "Maharashtra", 12)],
    [("10:00:00", "Karnataka", 15), ("10:00:00", "Maharashtra", 15),
     ("10:01:00", "Karnataka", 10), ("10:01:00", "Maharashtra", 15),
     ("10:02:00", "Maharashtra", 9)],
]

def demonstrar_streaming():
    resultados = []
    nome_consulta = "testes_ra2_" + uuid4().hex
    consulta = None
    # Criando uma pasta exclusiva, sem reaproveitar dados ou checkpoints.
    with tempfile.TemporaryDirectory(prefix="ra2_") as temporaria:
        raiz = Path(temporaria)
        entrada = raiz / "entrada"
        entrada.mkdir()
        try:
            # Lendo os novos arquivos que chegarem na pasta.
            fluxo = (spark.readStream.schema(schema_ra2)
                     .option("maxFilesPerTrigger", 1)
                     .option("mode", "FAILFAST")
```

```
                .json(str(entrada)))
            assert fluxo.isStreaming
            # Somando as amostras por estado em intervalos de 1 minuto.
            totais = (fluxo.groupBy(F.window("horario", "1 minute"), "estado")
                      .agg(F.sum("novas_amostras").alias("amostras")))
            consulta = (totais.writeStream.format("memory")
                        .queryName(nome_consulta).outputMode("complete")
                        .option("checkpointLocation", str(raiz / "checkpoint"))
                        .start())

            for numero, lote in enumerate(lotes_ra2, start=1):
                # Escrevendo fora da entrada e publicando o arquivo completo.
                arquivo = raiz / f"lote_{numero}.json"
                arquivo.write_text(
                    "\n".join(json.dumps(r) for r in lote) + "\n",
                    encoding="utf-8")
                arquivo.rename(entrada / arquivo.name)
                # Esperando o próprio Spark terminar os dados disponíveis.
                consulta.processAllAvailable()
                linhas = (spark.table(nome_consulta)
                          .orderBy("window.start", "estado")
                          .select(
                              F.date_format("window.start", "HH:mm:ss").alias("inicio"),
                              F.date_format("window.end", "HH:mm:ss").alias("fim"),
                              "estado", "amostras").collect())
                obtido = [(r.inicio, r.estado, r.amostras) for r in linhas]
                assert obtido == esperados_ra2[numero - 1], obtido
                recebidos = sum(p["numInputRows"] for p in consulta.recentProgress)
                assert recebidos == numero * 3, recebidos
                tabela = pd.DataFrame([tuple(r) for r in linhas], columns=[
                    "Início (UTC)", "Fim (UTC)", "Estado", "Novas amostras"])
                print(f"Após o lote {numero}: {recebidos} registros processados")
                display(tabela)
                resultados.append(tabela)
        finally:
            # Encerrando a consulta antes de apagar a pasta temporária.
            if consulta is not None:
                consulta.stop()
            spark.catalog.dropTempView(nome_consulta)
    print("Consulta encerrada. Os três lotes coincidem com os valores esperados.")
    return resultados

resultados_ra2 = demonstrar_streaming()
```

```
Após o lote 1: 3 registros processados
```

	Início (UTC)	Fim (UTC)	Estado	Novas amostras
0	10:00:00	10:01:00	Karnataka	8
1	10:00:00	10:01:00	Maharashtra	15

```
Após o lote 2: 6 registros processados
```

	Início (UTC)	Fim (UTC)	Estado	Novas amostras
0	10:00:00	10:01:00	Karnataka	15
1	10:00:00	10:01:00	Maharashtra	15
2	10:01:00	10:02:00	Karnataka	6
3	10:01:00	10:02:00	Maharashtra	12

```
Após o lote 3: 9 registros processados
```

	Início (UTC)	Fim (UTC)	Estado	Novas amostras
0	10:00:00	10:01:00	Karnataka	15
1	10:00:00	10:01:00	Maharashtra	15
2	10:01:00	10:02:00	Karnataka	10
3	10:01:00	10:02:00	Maharashtra	15
4	10:02:00	10:03:00	Maharashtra	9

```
Consulta encerrada. Os três lotes coincidem com os valores esperados.
```

Resultados confirmados na execução: no lote 1, a janela 10:00-10:01 soma 15 amostras em Maharashtra e 8 em Karnataka. No lote 2, Karnataka sobe para 15 nessa mesma janela, e surge a janela 10:01-10:02 com 12 e 6 amostras, respectivamente. No lote 3, essa segunda janela passa a 15 e 10, e aparece uma terceira janela com 9 amostras em Maharashtra.

Ao final, são 64 novas amostras simuladas : 39 em Maharashtra e 25 em Karnataka. Isso demonstra a atualização do volume de testagem por estado ao receber novos registros; não permite concluir nada sobre a pandemia real.

Para repetir: execute novamente resultados_ra2 = demonstrar_streaming() . A função cria uma nova entrada e um novo checkpoint e deve produzir as mesmas três tabelas. Os assert verificam valores por janela e o número de registros processados.

Roteiro de 5 minutos: apresente a pergunta e o caráter simulado (1 min), mostre os três lotes e o schema (1 min), explique a janela de 1 minuto (1 min), execute e compare as tabelas (1 min), destaque a validação e o encerramento (1 min).

Referência: Guia oficial do Spark Structured Streaming .

Validação da execução

O notebook completo foi executado em Linux pelo Docker, com Python 3.12, Java 17 e PySpark 3.5.7. O código do RA1 e os conteúdos dos três CSVs originais e dos três CSVs preparados permaneceram iguais.

O Spark recebeu 3, 6 e 9 registros acumulados , e os totais por estado e janela coincidiram com os valores esperados após cada lote. Os registros em 10:01:00 e 10:02:00 entraram nas novas janelas. O total final foi 64 novas amostras simuladas .

A simulação foi repetida no mesmo kernel: as três tabelas foram idênticas, usando novas pastas e checkpoints. Nenhuma consulta ficou ativa após o término. As tabelas acima são saídas reais do Spark. A execução nativa no Windows e a execução no Colab não foram validadas; o teste bem-sucedido foi no Docker/Linux.